

Congrats Congrats Money Money

"In this business it's easy to confuse luck with brains." - Jim Simons

Objective

Develop an end-to-end algorithmic trading pipeline for a universe of **N anonymized stocks**. Your goal is to transform raw price data into a trading system that maximizes risk-adjusted returns.

We are testing your ability to handle three critical stages of quantitative research: **Data Engineering, Strategy Formulation, and Simulation**.

General Instructions/Tips:

- There are **FOUR TOTAL parts to this task**.
 - a. Feature Engineering & Data Cleaning
 - b. Model Training & Strategy Formulation
 - c. Backtesting & Performance Analysis
 - d. Statistical Arbitrage Overlay
- We will expect **at least one Python notebook** as your deliverable for submission detailing every part of the three stage pipeline with comments and explanations of the approach taken.
- You may use Python scripts and multiple notebooks to modularize and clean up your code. We recommend this approach (ideally, three notebooks, supplementary scripts, and organized outputs).
- Do **make visuals** to illustrate your learnings and hypotheses wherever possible. It helps both you and us have a better time understanding it.
- **Learn math**. Nothing about this task is impossible to learn/implement with the knowledge a student from IIIT has acquired up until their 2nd year, first sem. It may seem daunting, but it's easier than it looks.
- This task is **open-ended**. Not everything will have a defined, objectively correct end goal. This is reflective of real quant research as well. Feel free to explore well beyond the scope of just this task's specifications.
- **KEEP LOOKING BACK AT THIS PAGE FOR POSSIBLE UPDATES**

Data

- **File:** `daily_prices.csv`, **T** years of OHLCV data for **N** tickers. (final file name may differ)
 - <https://www.kaggle.com/datasets/iamspace/precog-quant-task-2026>
- **Split:** The training, testing, and validations splits are entirely up to your creative process. However, keep in mind that we want to see your model perform out of sample for as long of a time frame as possible.
- **Note:** The years of data and number of stocks in the file that will be provided is open to change. This shouldn't affect how you approach the problem.

Part 1: Feature Engineering & Data Cleaning

Raw market data is messy and noisy. Show us how you handle it. Good data = good features, and good features = good model. (this prior statement has no mathematical backing and is not subject to any guarantees)

- **Cleaning:** Assess data quality. Handle missing values, outliers, or potential anomalies in the provided dataset.
- **Feature Extraction:** Generate features that capture market dynamics (e.g., momentum, volatility, volume patterns, or statistical factors). Get fancy with it. Think. What would actually be useful for a model to know?

Note: The quality of your inputs defines the ceiling of your performance. **Simple raw prices are rarely sufficient for high-quality predictions.** Too many features isn't a good thing either. Quant research is fast, so your models must be faster. *And remember, you are dealing with a universe here; not a single asset.*

Deliverable:

- A Python notebook showing your cleaning logic and feature generation code.

Part 2: Model Training & Strategy Formulation

Develop a predictive engine that translates your features from Part 1 into actionable trading signals for the universe.

- **Prediction:** Build a model (or models) to predict asset performance or rank attractiveness.
 - You have two broad choices for how you choose to define your prediction target:
 - Classification (whether the stock goes up, down, or stays the same)
 - Regression (predict, say, 1 day forward return, or 5 day forward return)
 - Regardless of what you choose, keep in mind that **you will eventually have to convert this prediction into a viable, tradeable signal** that your backtester is able to interpret.
- **Strategy Logic:** Define how these predictions translate into portfolio decisions.

Hint A: **Financial data is incredibly noisy.** Relying on a single signal source or a single naive model architecture can often lead to instability.

Hint B: **Markets evolve.** A relationship that held true in Year 1 may not exist in Year 3. Consider how your methodology ensures relevance as market dynamics shift over time.

Deliverable(s):

- A notebook detailing your modeling approach and the logic used to generate predictions.

Part 3: Backtesting & Performance Analysis

Simulate your strategy over the **2-Year Test Set**. Your backtester must realistically model the execution of your strategy.

- **Simulation Constraints:**
 - **Initial Capital:** \$1,000,000. (this isn't necessary, you can use whatever)
 - **Transaction Costs:** Deduct **0.10% (10 bps)** per trade. (play around with it, we also want to see performance in absence of transaction costs)
 - **Universe:** You may trade any subset of the **N** stocks.
- **Metrics:** Report the following metrics (Note that **all these metrics have multiple interpretations**. Your choice of interpretation is also open to you, but you will be expected to justify your choice in the context of your pipeline):
 - **Sharpe Ratio** (annualized)
 - **Maximum Drawdown** (and average)
 - **Portfolio Turnover**
 - **Return** (how much money we made 📈)

Deliverable(s):

- The backtesting code. We want to see out-of-sample performance on at least two years worth of data.
- A cumulative PnL plot (Strategy vs. Equal-Weight Benchmark). How much did your strategy make over a benchmark? (either the market or an equal-weight buy-and-hold portfolio of the same assets)
- A brief analysis: Did the strategy survive transaction costs? When and why did it fail?

Notes:

- We will be judging you based on the above metrics, but we will also be judging the period over which those metrics were delivered. A **lower Sharpe that remains stable over a long time frame is much more valuable than a ridiculously high Sharpe over one year.** *Always remember, you cannot see into the future.*
-

Part 4: Statistical Arbitrage Overlay

While the main pipeline focuses on broad alpha, specific opportunities often exist in relative value.

- Oftentimes, there exist assets that exhibit correlated or *cointegrated* movement: they move together. This movement may be instantaneous, or it may occur after a certain time lag.
- Your goal will be to find examples of such co-moving assets and explain the rationale behind their discovery.
- **Do not stick to pure correlation.** Use it as a baseline, but get creative with it. I won't provide any guidance or hints here as it would only serve to bias your research.
- Which assets seem correlated? Do they lie in the same sector? Is their customer base in the same country? Does sentiment play a role? Which timeframes do they appear to move together in? Which assets leads the relationship, and which asset lags in it?
- This is a very open ended question, and you may not even see statistically significant results. Do not be discouraged by this. It's all part of the quant research process. Make visuals, present ideas. We want it all.

Deliverable(s):

- **Analysis:** Present visuals and analysis of identified pairs or asset groups whose movements appear to be tethered together. Mathematically and empirically justify your approach and results.
- **Implementation Idea:** Demonstrate how you would incorporate this relative-value signal into your main portfolio structure. You don't need to code this up explicitly, but we would love to see mathematically backed ideas.

Something to consider before you dive in (feel free to skip)

- Quant research is messy. From start to finish, many things are abstracted away, and at times, the mathematical formulations used by researchers to try and make sense of things seem to only drive you further away from the truth. This is completely natural! Take it in stride as you proceed with this project.
- Don't be discouraged by the lack of results, and always be wary of results that seem too good.
- Remember that we cannot see into the future, and however elite your model is, it can't either.
- AI is your best friend that sometimes can't be trusted. Use it for your research, use it to learn, but always, always, validate what it's telling you.
- **Have fun with it!** Explore the data, make your visualizations, see how things move. An intuitive understanding only serves to supplement a purely mathematical one.
- The data is very naively anonymized. It's actually really easy to reverse engineer and figure out which asset's data it actually is. Feel free to try and come up with a mapping! It won't help you with this task regardless.

Doubts Document Link:

- <https://docs.google.com/document/d/1ybowflulkde2ggqlqVEkyZ84t4IpuDXBiX0Wk5ZOZs3E/edit?usp=sharing>